

OOP LAB 11

Problem Statements:

Task 1: Vehicle and Car Classes (Protected Access Specifier)

Vehicle.h

```
#include <string>
#include <iostream>
using namespace std;
class Vehicle {
private:
    string model;
    int fuelCapacity;
protected:
    // Function to check if vehicle is fuel efficient
    // Return true if fuel capacity > 50 liters
    bool isFuelEfficient(int capacity);

public:
    // Constructor with default values
    Vehicle(string m = "Unknown", int fc = 0);
    void displayVehicle();
    int getFuelCapacity();
};
```

Car.h

```
#include "Vehicle.h"
#include <string>
using namespace std;

class Car : public Vehicle {
private:
    string carType; // e.g., "Sedan", "SUV", "Hatchback"

public:
    // Constructor
    Car(string model, int fuelCapacity, string type);

    // Display car details including fuel efficiency check
    void showCarDetails();
};
```

Task 2: Appliance and WashingMachine (IS-A Relationship)

Appliance.h

```
#include <iostream>
using namespace std;
```

```
class Appliance {
public:
    // Base class function - turns on the appliance
    void turnOn();
};
```

WashingMachine.h

```
#include "Appliance.h"
using namespace std;
```

```
class WashingMachine : public Appliance {
public:
    // Derived class specific function - starts washing cycle
    void startWash();

    // Should be able to call base class functions
    void operateWashingMachine();
};
```

```
// Global function
```

```
void operateAppliance(Appliance &a);
```

Task 3: Product and DigitalProduct (Copy Constructor & Assignment Operator)

Product.h

```
#include <iostream>
#include <cstring>
using namespace std;
```

```
class Product {
private:
    char* name;
    float price;

public:
    Product(const char* n = "Unknown", float p = 0.0);
    Product(const Product& p);
    Product& operator=(const Product& p);
```

```

// Destructor - frees dynamically allocated memory
~Product();

// Display function - shows product name and price
void display() const;
const char* getName() const;
float getPrice() const;
void setName(const char* n);
void setPrice(float p);
};

```

DigitalProduct.h

```

#include "Product.h"
#include <cstring>
using namespace std;

class DigitalProduct : public Product {
private:
    char* fileFormat; // e.g., "PDF", "MP3", "EXE"

public:
    // Constructor with parameters for all data members
    DigitalProduct(const char* n, float p, const char* format);

    // Copy constructor
    DigitalProduct(const DigitalProduct& dp);

    // Assignment operator
    DigitalProduct& operator=(const DigitalProduct& dp);

    // Destructor - cleans up derived class resources
    ~DigitalProduct();

    // Display function - shows all product information including format
    void display() const override;

    // Getter for file format
    const char* getFormat() const;
};

```

Task 4: Implement a New Class Hierarchy (Function Overriding Without virtual)

You are required to implement a simple class hierarchy to understand how function overriding behaves when the base class does not use the virtual keyword. Create the following two classes:

```
class Shape {
public:
    void draw() {
        cout << "Drawing a generic shape" <<
endl;    }
};
class Circle : public
Shape { public:
    void draw() {
        // same name, but base class function is NOT virtual
cout << "Drawing a circle" << endl;
    }
};
```

Tasks

- a. Create an object of Shape and an object of Circle. Call draw() on both.
- b. Create a Shape* pointer that points to a Circle object.
Call draw() using the pointer and record the output.
- c. Modify only the Shape class by making draw() a virtual function. Run the same tests again and explain the difference in behavior.